```
      });

    await provider.executeTest(async mockProvider => {
      const movies = await fetchSingleMovie(mockProvider.url, testId);
      expect(movies).toEqual(EXPECTED_BODY);
    });
  });
});

describe('When a POST request is made to /movies', () => {
  test('it should add a new movie', async () => {

    const name = 'Harry Potter and the Philosopher\'s Stone';
    const year = 2001;

    EXPECTED_BODY.name = name;
    EXPECTED_BODY.year = year;

    provider
      .uponReceiving('a request to add a new movie')
      .withRequest({
        method: 'POST',
        body: {
          name,
          year,
        },
        path: '/movies',
      })
      .willRespondWith({
        status: 200,
        body: {
          id: integer(EXPECTED_BODY.id),
          name,
          year,
        }
      });

    await provider.executeTest(async mockProvider => {
      const movies = await addNewMovie(mockProvider.url, name, year);
      expect(movies).toEqual(EXPECTED_BODY);
    });
  });

  test('it should not add a movie that exists already', async () => {

    const name = 'Harry Potter and the Philosopher\'s Stone';
    const year = 2001;

    EXPECTED_BODY.name = name;
    EXPECTED_BODY.year = year;

    provider
      .given('an existing movie exists', { EXPECTED_BODY })
      .uponReceiving('a request to add an existing movie')
      .withRequest({
        method: 'POST',
        body: {
          name,
          year,
        },
        path: '/movies',
```